Grassroots Platforms with Atomic Transactions: Social Graphs, Cryptocurrencies, and Democratic Federations*

Ehud Shapiro[a,b,*]

[a] *Department of Mathematics and Data Science Institute, London School of Economics and Political Science, London, UK*
[b] *Department of Computer Science and Applied Mathematics, Weizmann Institute of Science, Rehovot, Israel*

Abstract

Grassroots platforms are person-to-person smartphone apps that realise Proudhon's two-centuries-old notion of a social contract that can have multiple independent instances that may coalesce by mutual consent, possibly into a single global instance. They aim to offer an egalitarian and democratic alternative to global platforms, centralised/autocratic (Facebook) and decentralised/plutocratic (Bitcoin). Key grassroots platforms include social graphs, social networks, cryptocurrencies, and democratically-governed federations. Previously, grassroots platforms were defined formally and proven grassroots using unary multiagent transition systems, in which each transition is carried out by a single agent. However, grassroots platforms cater for a more abstract specification using transactions carried out atomically by multiple agents, which unary transition systems cannot express, and their origi-

*A preliminary version of this paper appeared in the proceedings of ICDCN 2026 [1].
*Corresponding author.
Email address: e.shapiro2@lse.ac.uk (Ehud Shapiro)

nal specifications and proofs were unnecessarily cumbersome and opaque.

Here, we enhance the notion of a multiagent transition system to include atomic transactions and revisit the notion of grassroots platforms within this new foundation. We present concise specifications of key grassroots platforms using atomic transactions: befriending and unfriending for grassroots social graphs, coin swaps for grassroots cryptocurrencies, and communities forming, joining, and leaving a federation for grassroots federations. We prove a general theorem that a platform specified by atomic transactions that are interactive is grassroots; show that the atomic transactions specifying all three platforms are interactive; and conclude that the platforms are grassroots. We thus provide a crisp mathematical foundation for grassroots platforms and a clear starting point for their implementation.

Keywords: Grassroots protocols, multiagent transition systems, atomic transactions, social graphs, cryptocurrencies, democratic federations, peer-to-peer systems

---

1. Introduction

Background. The Internet today is dominated by centralised global platforms—social networks, Internet commerce, 'sharing-economy'—with autocratic control [2, 3]. An alternative is emerging—blockchains and cryptocurrencies [4–9]—that are also global platforms, but with decentralised, plutocratic control [10].

Grassroots platforms [11, 12] are person-to-person smartphone apps that realise Proudhon's two-centuries-old notion of a social contract [13, Sixth Study]: that (i) one could "make a contract with all, as ... with some" — a

grassroots platform can have multiple independent instances; and (*ii*) "each group of citizens ... formed by a like contract ... could thereafter, and always by a similar contract, agree with every and all other groups" — independent instances may coalesce by mutual consent, possibly into a single global instance. They aim to offer an egalitarian alternative to global platforms, centralised and decentralised alike. Global platforms can only have a single instance—one Facebook, one Bitcoin—as two instances of the platform would clash over the global resources they use—domain name, port number, or boot nodes, which are hardwired into their code. Even if it is possible to modify their code ('hard-fork') to create non-conflicting instances—Facebook′, Bitcoin′—such forked instances would ignore, rather than interoperate with, the primary instance. Grassroots platforms, in contrast, can have multiple instances that emerge and operate independently of each other and of any global resource except the network, yet can interoperate and coalesce once interconnected, forming ever-larger platform instances, potentially (but not necessarily) coalescing into a single instance.

Any platform that operates on a shared global resource or employs a single replicated (Blockchain [14]), or distributed (IPFS [15], DHT [16]) shared global data structure, as well as distributed pub/sub systems with a global directory [17–19], are all not grassroots. Mastodon [20] is similar in spirit but is peer-to-peer among servers, not people: a group of people with smartphones cannot do more if more people with smartphones join the group, unless they are also joined by a server, thus it does not fall under the definition of grassroots. BitTorrent [21] requires reaching a tracker or DHT boot nodes to join a swarm, and so is not grassroots either. An example of a

platform that is grassroots in spirit (even if not formally proven as such) is Scuttlebutt [22, 23].

Example: Grassroots social graphs. Each agent p maintains a set of friends $c_p \subseteq P$. The social graph evolves via befriending and unfriending, specified as binary atomic transactions:

1. **Befriend**: $c'_p := c_p \cup \{q\}$, $c'_q := c_q \cup \{p\}$, provided $q \notin c_p$ and $p \notin c_q$.
2. **Unfriend**: $c'_p := c_p \setminus \{q\}$, $c'_q := c_q \setminus \{p\}$, provided $q \in c_p$ and $p \in c_q$.

Each transaction has two participants, p and q, both of which change their local state atomically, and it can be carried out regardless of the local states of all other agents. This paper develops the framework in which such specifications are written, and proves that platforms so specified are grassroots.

Motivation. Key grassroots platforms include grassroots social graphs and networks [24], grassroots cryptocurrencies [25, 26], and grassroots federations [27, 28]. Previously, grassroots platforms were defined formally using unary multiagent transition systems, in which each transition is carried out by a single agent [11]. However, grassroots platforms cater for a more abstract specification using transactions carried out atomically by multiple agents, something that cannot be expressed by unary transition systems. As a result, their original specifications were more cumbersome and opaque than they should be. Moreover, in grassroots platforms transactions carried out by different sets of participants need not be synchronised with each other, which greatly simplifies their mathematical foundations.

Here, we aim to provide a crisp formal foundation for these key grassroots platforms and beyond. To do so, we enhance the notion of a multiagent transition system to include atomic transactions and revisit the notions of grassroots platforms and their implementation within this enhanced foundation.

Furthermore, previous work [11] provided sufficient conditions for when a platform (formally, a protocol defined via a family of multiagent transition systems) is grassroots. While going through the route presented in previous work is possible, the new foundations offer a simpler, more direct and mathematically preferable way to prove that a platform, specified via atomic transactions, is grassroots. Here, we follow this simpler path.

Key Grassroots Platforms. Three exemplar grassroots platforms motivate our work: grassroots social graphs, where people maintain their friendship connections through local storage and peer-to-peer relationships without central control; grassroots cryptocurrencies, where individuals mint personal coins and establish mutual credit lines through atomic swaps; and grassroots federations, where communities form and federate democratically without central coordination. Section 5 presents their detailed specifications using atomic transactions.

Paper outline. Section 2 introduces multiagent atomic transactions, their participants, active and stationary, and concise sets of transactions. Section 3 introduces multiagent transition systems and how they are induced by a set of atomic transactions. Section 4 introduces protocols and grassroots protocols; shows how a set of atomic transactions induces a protocol; defines when a set of atomic transactions is interactive; and proves the main theo-

rem, that a protocol induced by an interactive set of atomic transactions is grassroots. Section 5 presents the transactions-based specifications of three key grassroots platforms—grassroots social graphs, grassroots cryptocurrencies, and grassroots federations—proves their invariants, and proves, via the main theorem, that all three are grassroots. Section 6 discusses, informally, the implementation of grassroots platforms. Section 7 discusses related and future work and concludes.

2. Multiagent Atomic Transactions

Grassroots protocols were defined formally using unary multiagent transition systems [11]. Thus, higher-level protocols that include atomic transactions by multiple agents could not be expressed directly within this formalism, let alone proven grassroots. Yet, the grassroots platforms we are interested in—grassroots social graphs, grassroots cryptocurrencies, and grassroots federations—all call for atomic transactions for their specification. Hence this paper. Here, we use atomic transactions carried out by a set of agents as the basis for specifying grassroots platforms.

Agent, state, configuration. We assume a potentially infinite set of *agents* Π (think of all the agents that are yet to be born), but consider only finite subsets of it, so when we refer to a particular set of agents $P \subset \Pi$ we assume P to be nonempty and finite. We use $\subset$ to denote the strict subset relation and $\subseteq$ when equality is also possible, and use $p \neq q \in P$ as a shorthand for $p \in P \wedge q \in P \wedge p \neq q$.

In the context of multiagent systems it is common to refer to the state of the system as *configuration*, so as not to confuse it with the *local states* of

the agents. As standard, we use S^P to denote the set of all total functions from P to S, and if $c \in S^P$ we use c_p (instead of $c(p)$) to denote the value of c at $p \in P$. Intuitively, think of such a $c \in S^P$ as an array of cells indexed by members of P and with cell values in S.

Atomic transactions. Informally, an atomic transaction:

1. Can have several participants (be binary, k-ary), the active ones among them changing their local states atomically as specified.
2. Specifies explicitly its participants (both active and stationary), thus implicitly defining infinitely-many multiagent transitions where all non-participants remain in their arbitrary state.

In addition to being atomic, the description above implies that transactions are asynchronous [29], in the sense that if a transaction can be carried out by its participants, with non-participants being in any arbitrary states, it can still be carried out no matter what the non-participants do. In particular, the active participants in a transaction need not synchronise its execution with any non-participant (but they must synchronise, of course, with stationary participants, as changing the local state of any participant would disable the transaction).

Definition 2.1 (Local States, Configuration, Transaction, Active & Stationary Particip

Given agents $Q \subset \Pi$ and an arbitrary set S of **local states**, a **configuration** over Q and S is a member of $C := S^Q$. A **multiagent atomic transaction**, or just **transaction**, over **participants** Q and S is any pair of configurations $t = c \to c' \in C^2$ s.t. $c \neq c'$, with $t_p := c_p \to c'_p$ for any $p \in Q$, and with p being an **active participant** in t if $c_p \neq c'_p$, **stationary participant** otherwise. A transaction is **unary** if $|Q| = 1$.

Active and stationary participants. The participants of a transaction are the agents whose local states the transaction reads. An active participant changes its local state when the transaction is taken; a stationary participant does not, but its local state is a precondition of the transaction all the same. In the befriend transaction of the Introduction the participants are p and q, and both are active. Every transaction of the three platforms of Section 5 is of this kind: all its participants are active. A stationary participant arises when a transaction is conditioned on the state of an agent it does not change. For example, in a child-safe social network [30], two children r and s with respective parents p and q (four distinct agents) may befriend only if their parents are friends:

1. **Child befriend**: $c'_r := c_r \cup \{s\}$, $c'_s := c_s \cup \{r\}$, provided $q \in c_p$, $p \in c_q$, $s \notin c_r$ and $r \notin c_s$.

Its participants are r, s, p and q; the children r and s are active, and the parents p and q are stationary: the precondition $q \in c_p$, $p \in c_q$, that the parents are friends, reads their local states, which the transaction does not change.

Degree. The *degree* of a transaction is its number of participants, active and stationary alike. Befriend and unfriend are binary; child befriend is quaternary, with two active and two stationary participants; and the transaction by which an agent mints coins (Section 5.2) is unary.

Redundant participants and concise specifications. A stationary participant in a transaction is redundant if the active participants can ignore its state when performing the transaction. For example, if p and q can befriend

regardless of r's state, then including r as a stationary participant would be redundant—it obscures the transaction's essential coordination requirements. Specifications of grassroots platforms using transactions should therefore be concise, namely harbour no redundant participants. The parents in child befriend are stationary but not redundant: the transaction cannot be taken when a parent's local state does not record the other parent as a friend.

Definition 2.2 (Degree, Redundant Participant, Concise Set of Transactions). Given agents $P \subset \Pi$, local states S, and a set of transactions T, each $t \in T$ over some $Q \subseteq P$ and S, the **degree** of $t \in T$ (unary, binary,... k-ary) is the number of participants in t, and the **degree** of T is the maximal degree of any $t \in T$. A stationary participant $q \in Q$ in a transaction $t \in T$ over Q is **redundant** in t (given T) if for $\forall s \in S\ \exists t' \in T$ such that $t'_q = s \to s$ and $t_p = t'_p$ for every $p \neq q \in Q$. A set of transactions is **concise** if it has no transactions with redundant participants.

Every transaction of the three platforms specified in Section 5 has active participants only, so the sets of transactions specifying them are concise.

3. Multiagent Transition Systems

In this section we define transition systems and multiagent transition systems, and show how a set of atomic transactions induces a multiagent transition system.

3.1. Transition systems

The following is a simplified variation, sufficient for the purpose of this work, on the foundations introduced in [29].

Definition 3.1 (Transition system, Computation, Run). A **transition system** is a tuple $TS = (S, s_0, T)$, where:

1. S is an arbitrary non-empty set, referred to as the set of **states**.
2. Some $s_0 \in S$ is the designated **initial state**.
3. $T \subset S^2$ is a set of **transitions over** S, where each transition $t \in T$ is a pair (s, s') of non-identical states $s \neq s' \in S$, also written as $t = s \to s'$.

A **computation** of TS is a (nonempty, potentially infinite) sequence of states $r = s_1, s_2, \cdots$ such that for every two consecutive states $s_i, s_{i+1} \in r$, $s_i \to s_{i+1} \in T$. If $s_1 = s_0$ then the computation is called a **run** of TS.

Given a computation $r = s_1, s_2, \ldots$, we use $r \subseteq T$ to mean $(s_i \to s_{i+1}) \in T$ for every $(s_i \to s_{i+1}) \in r$.

We note that the notion of transition systems originally employed to define grassroots systems [11, 29] included a liveness condition. The abstract atomic transactions employed here are volitional and have no associated liveness conditions; adding these is discussed in Section 7. Also, the original work considered faulty computations and fault-tolerant implementations. These could be re-introduced in follow-on work that considers live and fault-tolerance implementations of the specifications presented here.

3.2. Multiagent Transition Systems with Atomic Transactions

Definition 3.2 (Multiagent Transition System). Given agents $P \subset \Pi$ and an arbitrary set S of **local states**, a **multiagent transition system** over P and S is a transition system $TS = (C, c_0, T)$ with **configurations** $C := S^P$, a designated **initial configuration** $c_0 \in C$, and **transitions**

$T \subseteq C^2$ being a set of transactions over P and S, with the **degree** of TS being the degree of T.

Typically, the initial configuration assigns every agent the same *initial local state* $s_0 \in S$, $c_0 = \{s_0\}^P$; in grassroots federations (Section 5.3) the initial local state of an agent names the agent. Unary multiagent transition systems were introduced in [29] and were employed to define the notion of grassroots protocols [11] and to provide unary specifications for various grassroots platforms [24–26]. Here, we employ k-ary transition systems, for any $k \leq |P|$, in which several agents can change their state simultaneously.

However, rather than specifying a multiagent transition system over a set of agents P directly, we specify it via the concise atomic transactions (Definitions 2.1, 2.2) the transition system intends to realise, which are typically of bounded degree, or at least a degree smaller than P.

A transaction over $Q \subseteq P$ defines a set of multiagent transitions over P in which all members of $P \setminus Q$ are stationary:

Definition 3.3 (Transaction Closure). Let $P \subset \Pi$, S a set of local states, and $C := S^P$. For any transition or transaction $t = c \to c'$, we write $t_q := c_q \to c'_q$ and say p is **stationary** in t if $c_p = c'_p$. For a transaction $t = (c \to c')$ over local states S with participants Q, the **P-closure of** t, $t{\uparrow}P$, is the set of transitions over P and S defined by:

$$t{\uparrow}P := \begin{cases} \{t' \in C^2 : \forall q \in Q.(t_q = t'_q) \wedge \forall p \in P \setminus Q.(p \text{ is stationary in } t')\} & \text{if } Q \subseteq P \\ \emptyset & \text{otherwise} \end{cases}$$

If R is a set of transactions, each $t \in R$ over some Q and S, then the **P-**

closure of R, $R{\uparrow}P$, is the set of transitions over P and S defined by:

$$R{\uparrow}P := \bigcup_{t \in R} t{\uparrow}P$$

A transition over P is **unary** if it lies in the P-closure of a unary transaction.

Namely, the closure over $P \supseteq Q$ of a transaction t over Q includes all transitions t' over P in which members of Q do the same in t and in t', and the rest remain in their current (arbitrary) state. If distinct transactions in R have disjoint P-closures—as when every participant of every transaction in R changes state—then each transition in $R{\uparrow}P$ has a unique inducing transaction.

Note that while the set of multiagent transitions $R{\uparrow}P$ in general may be over a larger set of agents than R, $R{\uparrow}P$ and R are of the same degree, by construction. Also, note that while all transitions of a multiagent transition system over P are also over P, each transaction in a set of transactions is typically over a different set of participants. Thus, a set of transactions R over S, each with participants $Q \subseteq P$, defines a multiagent transition system over S and P as follows:

Definition 3.4 (Transactions-Based Multiagent Transition System). Given agents $P \subset \Pi$, local states S, an initial configuration $c_0 \in S^P$, and a set of transactions R, each $t \in R$ over some $Q \subseteq P$ and S, the **transactions-based multiagent transition system** over P, S, c_0 and R is the multiagent transition system $TS = (S^P, c_0, R{\uparrow}P)$.

In other words, one can fully specify a multiagent transition system over S and P simply by providing a concise set of atomic transactions over S, each

with participants $Q \subseteq P$. This is what we do in Section 5 for grassroots social graphs, grassroots cryptocurrencies, and grassroots federations.

4. Grassroots Protocols

Here we define what is a protocol; define when a protocol is grassroots; show how to define a protocol via a set of transactions; present conditions under which a protocol defined via transactions is grassroots; and, in Section 5, argue that the three protocols defined via transactions there satisfy these conditions, and conclude that these three protocols are grassroots.

4.1. Protocols and Grassroots Protocols

A protocol is a family of multiagent transition systems, one for each set of agents $P \subset \Pi$, which share an underlying set of local states $\mathcal{S}$ with a designated initial local state $s_0(p) \in \mathcal{S}$ for each agent $p \in \Pi$. A *local-states function* $S : P \mapsto 2^{\mathcal{S}}$ maps every set of agents $P \subset \Pi$ to an arbitrary set of local states $S(P) \subset \mathcal{S}$ that includes $s_0(p)$ for every $p \in P$ and satisfies $P \subset P' \subset \Pi \implies S(P) \subset S(P')$. Given a local-states function S, we use C to denote configurations over S, with $C(P) := S(P)^P$ and $c_0(P) \in C(P)$ defined by $c_0(P)_p := s_0(p)$ for every $p \in P$; in particular $c_0(P)_p = c_0(P')_p$ for every $p \in P \subset P'$. Next, we define the notions of a protocol and a grassroots protocol.

Definition 4.1 (Protocol). A **protocol** $\mathcal{F}$ over a local-states function S is a family of multiagent transition systems that has exactly one transition system $\mathcal{F}(P) = (C(P), c_0(P), T(P))$ for every $P \subset \Pi$.

Informally, in a grassroots protocol a set of agents P, if embedded within a larger set P', can still behave as if it is on its own, but has additional behaviours at its disposal due to possible interactions with members of $P' \setminus P$. To define the notion formally we employ the notion of projection.

Definition 4.2 (Projection). Let $\emptyset \subset P \subset P' \subset \Pi$. If c' is a configuration over P' then c'/P, the **projection of** c' **over** P, is the configuration c over P defined by $c_p := c'_p$ for every $p \in P$.

Note that in the definition above, c_p, the state of p in c, is in $S(P')$, not in $S(P)$, and hence may include elements that no run of P on its own could produce, e.g., friendship with or a coin of $q \in P' \setminus P$.

We use the notions of projection and closure (Definition 3.3) to define the key notion of this paper, a grassroots protocol:

Definition 4.3 (Oblivious, Interactive, Grassroots). A protocol $\mathcal{F}$ is:

1. **oblivious** if for every $\emptyset \subset P \subset P' \subseteq \Pi$, $T(P)\uparrow P' \subseteq T(P')$
2. **interactive** if for every $\emptyset \subset P \subset P' \subseteq \Pi$ and every configuration $c \in C(P')$ such that $c/P \in C(P)$, there is a computation $c \xrightarrow{*} c'$ of $\mathcal{F}(P')$ for which $c'/P \notin C(P)$.
3. **grassroots** if it is oblivious and interactive.

Oblivious. Being oblivious implies that if a run of $\mathcal{F}(P')$ reaches some configuration c', then anything P could do on their own in the configuration c'/P (with a transition from $T(P)$), they can still do in the larger configuration c' (with a transition from $T(P')$), effectively being oblivious to members of

$P' \setminus P$. The reason is that if $t = (c'/P \to d) \in T(P)$, then by the definition of closure, $t' = (c' \to d') \in t{\uparrow}P' \subseteq T(P')$, where $d'_p = d_p$ if $p \in P$ else $d'_p = c'_p$. Inductively, this implies that if agents P employ only transitions in $T(P)$ from the start, with their local states remaining in $S(P)$, they could continue doing so indefinitely, effectively ignoring any members in $P' \setminus P$. (Proving the corresponding claim within the original definition [29] was more difficult, and required the notion of interleaving of computations of two multiagent transition systems.)

We note that any protocol that uses a global data structure, whether replicated (Blockchain) or distributed (DHT [16], IPFS [15]), is not oblivious as members of P cannot ignore changes to the global data structure made by members of $P' \setminus P$; and hence is not grassroots.

Interactive. A slight complication in the definition of interactive is the use of a finite computation $\xrightarrow{*}$ rather than just a single transition $\to$. It is required as in some grassroots platforms agents need to make some preparatory steps before they may interact with each other. For example, in grassroots cryptocurrencies agents need first to mint some coins before they can swap them.

Being interactive is a weak liveness requirement. Informally, a standard liveness requirement has the form "something good must eventually happen". Here, the requirement is "it must be the case that something good (P interacting with non-P) may eventually happen". Namely, no matter what members of P do, if they run within a larger set of agents it is always the case that they can eventually interact with non-P's. Moreover, being interactive not only requires that P can always eventually interact with $P' \setminus P$,

but that they do so in a way that leaves a trace in the local states of P: the resulting configuration c'/P is not a configuration over P and $S(P)$, and so could not have been produced by P running on their own. For example, forming friendships with agents outside of P results in their non-P names entering c/P; receiving coins from agents outside of P results in their non-P coins entering c/P. We note that while federated systems [20] are oblivious, as one server may choose to ignore all other servers and just serve its clients, they are not interactive, as a set of clients P without a server cannot do more if embedded within a larger set of clients P', still without a server.

4.2. Transactions-Based Grassroots Protocols

The original grassroots paper [11] defined grassroots protocols over unary multiagent transition systems and provided three sufficient conditions for a protocol to be grassroots—asynchrony, interference-freedom, and interactivity. There, the local states of a protocol carry a partial order in which every transition is monotonic—think of a growing set of blocks or a growing sequence of messages—and a protocol is *asynchronous* if a p-transition enabled in a configuration remains enabled in every larger configuration with the same local state of p, so that progress by other agents cannot disable it. It is *non-interfering* if a transition of a group P is also a transition of any larger group P' whose other members are in their initial state, and if an agent live in a run of P remains live when the run is extended by transitions of $P' \setminus P$; and it is *interactive* if the projection on P of the transition system of P' has a transition that the transition system of P does not. Proving these for a specific protocol required, for each of them, an argument about its runs; and the monotonicity that asynchrony presupposes excludes transitions that

remove elements from a local state, such as unfriending or swapping coins.

They should still hold under the new definition, with some adaptation, mostly simplification. However, here we are interested in transactions-based transition systems, therefore we will follow a different route: We first show how a local-states function can be used to define a set of transactions, and then show how a set of such transactions can be used to define a protocol, termed transactions-based protocol. We then prove that a single condition on such transactions—interactivity—is sufficient for a transactions-based protocol to be grassroots.

Definition 4.4 (Transactions Over a Local-States Function). Let S be a local-states function. A set of transactions R is **over** S if every transaction $t \in R$ is a multiagent transition over Q and $S(Q')$ for some $Q \subseteq Q' \subset \Pi$. Given such a set R and $P \subset \Pi$, $R(P) := \{t \in R : t \text{ is over } Q \text{ and } S(Q'), Q \subseteq Q' \subseteq P\}$.

Definition 4.5 (Transactions-Based Protocol). Let S be a local-states function and R a set of transactions over S. Then a **protocol $\mathcal{F}$ over R and S** includes for each set of agents $P \subset \Pi$ the transactions-based multiagent transition system $\mathcal{F}(P)$ over P, $S(P)$, $c_0(P)$ and $R(P)$, $\mathcal{F}(P) := (S(P)^P, c_0(P), R(P)\uparrow P)$.

Next we aim to find conditions under which a transactions-based protocol is grassroots. In the original paper [11], fulfilling three conditions were deemed sufficient: Asynchrony, interference-freedom, and interactivity. Intuitively speaking, transactions-based multiagent transition systems are asynchronous by construction, as the essence of a transaction is that it can be taken no

matter what the states of non-participants are. They are also non-interfering for the same reason: the closure of a transaction (Definition 3.3) includes a transition for every configuration of the non-participants, whatever their local states, so a transition of P lifted to P' with the members of $P' \setminus P$ stationary—in their initial state, as non-interference requires, or in any other—is a transition of P'. No order on local states is needed, and there is no liveness requirement to interfere with. The following Proposition states this:

Proposition 4.6. *A transactions-based protocol is oblivious.*

To prove it, we need the following Lemma:

Lemma 4.7 (Closure Transitivity). *Let $\emptyset \subset P \subset P' \subset \Pi$ and R a set of transactions. Then*

$$(R(P)\uparrow P)\uparrow P' = R(P)\uparrow P'.$$

Proof. We argue both directions of the equality:

1. $\subseteq$: Let $t \in R(P)$ and consider any $t' \in (t\uparrow P)\uparrow P'$. By definition, $t_p = t'_p$ if $p \in P$, p is stationary in t' if $p \in P' \setminus P$, which, by the definition of closure, $t' \in t\uparrow P'$, namely $t' \in R(P)\uparrow P'$.
2. $\supseteq$: Let $t' \in R(P)\uparrow P'$. Then there is a transaction $\hat{t} \in R(P)$ over some $Q \subseteq P$, for which $t' \in \hat{t}\uparrow P'$. By construction, $\hat{t}_p = t'_p$ if $p \in Q$ and p is stationary in t' if $p \in P' \setminus Q$. By definition of closure, $t = \hat{t}\uparrow P$ satisfies $t' \in t\uparrow P'$, thus $t' \in (\hat{t}\uparrow P)\uparrow P'$, namely $t' \in (R(P)\uparrow P)\uparrow P'$.

We can now prove the Proposition:

PROOF (PROOF OF PROPOSITION 4.6). Let $\mathcal{F}$ be a protocol over the state function S, R a set of transactions over S, and $\emptyset \subset P \subset P' \subseteq \Pi$. We have to show that $T(P)\uparrow P' \subseteq T(P')$. By definition $T(P) = R(P)\uparrow P$ and $T(P') = R(P')\uparrow P'$, thus:

$$T(P)\uparrow P' = (R(P)\uparrow P)\uparrow P' = R(P)\uparrow P' \subseteq R(P')\uparrow P' = T(P').$$

by definition of $T(P)$, Lemma 4.7, and since $P \subset P'$ and therefore $R(P) \subseteq R(P')$.

The remaining condition is being interactive, which we aim to capture as follows:

Definition 4.8 (Interactive Transactions). A set of transactions R over a local-states function S is **interactive** if for every $\emptyset \subset P \subset P' \subset \Pi$ and every configuration $c \in C(P')$ such that $c/P \in C(P)$, there is a computation $(c \xrightarrow{*} c') \subseteq R(P')\uparrow P'$ for which $c'/P \notin C(P)$.

In other words, with an interactive set of transactions, any group of agents that have been so far self-contained will always have a computation with non-members that results in members of the group having a local state outside $S(P)$, one that could have been produced only by interacting with non-members.

Proposition 4.9. *A protocol over an interactive set of transactions is interactive.*

PROOF. Let S be a local-states function, R an interactive set of transactions over S, and $\mathcal{F}$ a transactions-based protocol over R and S, $\emptyset \subset P \subset P' \subseteq \Pi$,

and $c \in C(P')$ a configuration such that $c/P \in C(P)$. By R being interactive (Definition 4.8), there is a computation $c \xrightarrow{*} c' \subseteq R(P')\uparrow P'$ for which $c'/P \notin C(P)$. By the definition of $\mathcal{F}$ as a transactions-based protocol over R and S, $T(P') = R(P')\uparrow P'$, hence this computation is of $T(P')$, establishing that $\mathcal{F}$ is interactive.

Our main result follows from the definitions and results above:

Theorem 4.10. *A protocol over an interactive set of transactions is grassroots.*

PROOF. Let $\mathcal{F}$ be a protocol over a set of transactions R (Definition 4.5), where R is interactive (Definition 4.8). Since $\mathcal{F}$ is a transactions-based protocol then, according to Proposition 4.6, $\mathcal{F}$ is oblivious. And since $\mathcal{F}$ is over an interactive set of transactions then, according to Proposition 4.9, $\mathcal{F}$ is interactive. Therefore, by Definition 4.3, $\mathcal{F}$ is grassroots.

5. Three Grassroots Platforms

We now present concise specifications of three grassroots platforms using atomic transactions. We revisit the grassroots platforms previously defined using unary transition systems—social graphs and cryptocurrencies—and redefine them more simply and abstractly using binary atomic transactions. In addition, we present a k-ary transactions-based specification of grassroots federations, which were hitherto defined only at the community level. For each platform, we specify its atomic transactions, labelling their participants; define the transactions-based protocol they induce (Definition 4.5); prove its invariants, properties preserved across all runs of the induced multiagent

transition systems, which are the platform-specific analogue of safety in the classical sense; and prove that the platform is grassroots via Theorem 4.10. In all three platforms every participant of every transaction is active, so the sets of transactions are concise and distinct transactions have disjoint closures.

5.1. Grassroots Social Graphs via Befriending and Unfriending

Grassroots social graphs are the foundational infrastructure for grassroots platforms, including grassroots social networks which provide Twitter/X-like feeds and WhatsApp-like groups without subjecting people or their social graph to control and exploitation. In centralised platforms (Twitter, Facebook, WhatsApp) the social graph is stored, controlled, and commercially exploited [2, 3] by the central authority. In proposed decentralised architectures [31], it resides on the globally-replicated blockchain under the control of miners/validators who are remunerated for their service.

In a grassroots social network, the social graph is stored in a distributed way under the control of the people themselves, with each person storing the local neighbourhood pertaining to them, and no third-party having access unless explicitly granted. The social graph evolves by people forming and breaking friendships, establishing authenticated peer-to-peer connections. The original definition [24] was via a unary multiagent transition system. Here both actions are specified—abstractly and concisely—as binary transactions.

Each agent maintains, as its local state, a set of friends; two agents p and q can atomically become friends, if they were not friends beforehand; and the two friends p and q can atomically cease to be friends. Communication

functions of a social network can be added, under the restriction that communication occurs only among friends [24]. For example, feeds with followers, where friends follow each other, and if two friends follow a third person, then the first to obtain an item on the third person's feed disseminates it to the other. Formally, a liveness theorem can be proven for this design [24], stating that if a person p that follows a person q is connected to q via a chain of mutual friends, each of them correct and follows q, then p will eventually receive every item on q's feed. Practically, a true "network celebrity" employing this protocol may achieve efficient large-scale distribution of their feed via the friendship subgraph of their followers. The specification of the grassroots social graphs is the foundation for grassroots social networks with feeds, groups, messaging, explored elsewhere [24, 32].

Transactions. The grassroots social graph transactions are the befriend and unfriend transactions of the Introduction, repeated here:

1. **Befriend**: $c'_p := c_p \cup \{q\}$, $c'_q := c_q \cup \{p\}$, provided $q \notin c_p$ and $p \notin c_q$.
2. **Unfriend**: $c'_p := c_p \setminus \{q\}$, $c'_q := c_q \setminus \{p\}$, provided $q \in c_p$ and $p \in c_q$.

Both are binary transactions with participants $\{p, q\}$, both of which are active. Formally:

Definition 5.1 (Grassroots Social Graphs Transactions). For agents $P \subset \Pi$ with local states $S(P) := 2^P$, the social graphs transactions are:

1. **Befriend**: A binary transaction $c \to c'$ with participants $\{p, q\} \subseteq P$ where $c'_p := c_p \cup \{q\}$ and $c'_q := c_q \cup \{p\}$, provided $q \notin c_p$ and $p \notin c_q$

2. **Unfriend**: A binary transaction $c \to c'$ with participants $\{p, q\} \subseteq P$ where $c'_p := c_p \setminus \{q\}$ and $c'_q := c_q \setminus \{p\}$, provided $q \in c_p$ and $p \in c_q$

Definition 5.2 (Grassroots Social Graphs). The **grassroots social graphs** SG is the transactions-based protocol (Definition 4.5) over the befriend and unfriend transactions of Definition 5.1, with local-states function $S(P) := 2^P$ and initial local state $s_0(p) := \emptyset$ for every $p \in \Pi$.

Invariants.

Lemma 5.3 (Mutuality). *Given $P \subset \Pi$, a run r of $SG(P)$, a configuration $c \in r$, and agents $p, q \in P$, then $p \in c_q$ iff $q \in c_p$.*

Proof. The proof is by induction on the length of the run $r = c_0, c_1, \ldots, c_n$. Assume $|r| = 1$, namely r consists of the initial configuration c_0. Then all local states are empty, satisfying the Lemma. Assume the lemma holds for runs of length $|r| = n$, that c_n satisfies the Lemma, and consider the transition $c_n \to c_{n+1}$. It can be either Befriend or Unfriend; both satisfy for every $p \in P$ the condition $p \in c_q$ iff $q \in c_p$ for c_{n+1} if c_n does.

We note that each configuration c in a run r of $SG(P)$ induces a graph with agents as vertices and an edge $p \leftrightarrow q$ if $p \in c_q$ and $q \in c_p$, and that the graphs induced by two consecutive configurations in r differ by exactly one added or removed edge.

Grassroots.

Corollary 5.4. *Grassroots Social Graphs are grassroots.*

PROOF. Consider $P \subset P'$ and a configuration $c \in C(P')$ such that $c/P \in C(P)$. This means that all friendships of members of P in c are with other members in P. Hence the transaction in $R(P')$ in which a member of P establishes friendship with a member of $P' \setminus P$ satisfies the definition of interactivity, and SG is grassroots by Theorem 4.10.

5.2. Grassroots Cryptocurrencies via Atomic Swaps

Grassroots cryptocurrencies [25, 26] can provide a foundation for an equitable digital economy, where people price their goods and services in terms of their own personally-minted grassroots coins, and liquidity is achieved through the creation of mutual credit lines among persons (natural and legal) via the swap of personal coins. Grassroots coins issued by different persons form an integrated digital economy via the sole rule in the grassroots cryptocurrencies protocol:

Coin Redemption: p can redeem a q-coin it holds against any coin held by q, one-for-one.

We note that the value of p-coins critically depends on p maintaining computational and economic integrity. In the specification of grassroots cryptocurrencies each agent maintains, as its local state, the set of coins they hold; each agent p may mint additional p-coins ($¢_p^k$ denotes k p-coins); and two agents p and q can atomically swap any coins they hold.

The original specification of grassroots cryptocurrencies was via unary multiagent transition system and hence was quite involved, and so was the proof of them being grassroots [25]. The definition in turn led to an implementation via a unary payment system [26]. Here we provide an alternative

specification—abstract and concise—via one unary and one binary transaction. We will later prove the specification to be grassroots.

Informally, we associate with each agent $p \in P$ a unique 'colour' and an infinite multiset of identical p-coloured coins. When applied to multisets of coins, $\cup$ and $\setminus$ denote multiset union and multiset difference, respectively. Formally:

Definition 5.5 (Coins). We assume an infinite multiset C of identical **coins** ¢. Given set of agents $P \subset \Pi$, we refer to $¢_p$, a p-indexed element of the indexed set C^P, as a p-coin, with $¢_p^k$ being a multiset of k p-coins. A **set of P-coins** is a member of 2^{C^P}, namely a multiset of P-indexed coins.

Transactions. The transactions of grassroots cryptocurrencies include the unary minting of p-coins by p, and the atomic swap between p and q of a set of coins held by p in exchange for a set of coins held by q:

1. **Mint**: $c'_p := c_p \cup ¢_p^k$, $k > 0$.
2. **Swap**: $c'_p := (c_p \cup y) \setminus x$, $c'_q := (c_q \cup x) \setminus y$, provided $x \subseteq c_p$ and $y \subseteq c_q$.

Mint is unary, with the single active participant p; Swap is binary, with participants $\{p, q\}$, both active. Formally:

Definition 5.6 (Grassroots Cryptocurrencies Transactions). For agents $P \subset \Pi$ with local states $S(P) := 2^{C^P}$, the grassroots cryptocurrencies transactions are:

1. **Mint**: A unary transaction $c \to c'$ with participant $p \in P$ where $c'_p := c_p \cup ¢_p^k$ for $k > 0$

2. **Swap**: A binary transaction $c \to c'$ with participants $\{p, q\} \subseteq P$ where $c'_p := (c_p \cup y) \setminus x$ and $c'_q := (c_q \cup x) \setminus y$, provided $x \subseteq c_p$ and $y \subseteq c_q$

The swap transaction can realise the key economic functions of grassroots cryptocurrencies:

1. **Payment**: Paying q with q-coins, namely $x = ¢_q^j$, $j > 0$, and $y = \emptyset$ (also paying with other coins q accepts as payment, e.g. community-bank coins). A payment could be made for love or, more typically, expecting in exchange (or after having received) an 'off-chain' product or service from q. In a practical application y could include a payment receipt, a confirmed purchase order, etc.
2. **Mutual Credit Lines**: Establish mutual credit lines via the swap of personal (self-minted) coins, namely $x = ¢_p^j$ and $y = ¢_q^k$ for some $j, k > 0$. Typically $j = k$, but mutual credit with a premium for one of the agents, namely with $j \neq k$, is also possible. A digital economy based on grassroots cryptocurrencies achieves liquidity through persons (natural and legal) establishing mutual credit lines among them.
3. **Redemption**: The obligatory 1:1 swap of q-coins held by p against an equal number of coins held by q, namely $x = ¢_q^j$, $|y| = j$.

The present specification does not distinguish between voluntary swap transactions, namely payments and forming mutual credit, and obligatory swaps, namely coin redemption. This issue is further discussed below.

Definition 5.7 (Grassroots Cryptocurrencies). The **grassroots cryptocurrencies** GC is the transactions-based protocol (Definition 4.5) over

the Mint and Swap transactions of Definition 5.6, with local-states function $S(P) := 2^{C^P}$ and initial local state $s_0(p) := \emptyset$ for every $p \in \Pi$.

Invariants.

Lemma 5.8 (Conservation of Money). *Given $P \subset \Pi$, a run r of $GC(P)$, a configuration $c \in r$ and an agent $p \in P$, the number of p-coins in c is the number of p-coins minted by p in the prefix of r ending in c.*

PROOF. The proof is by induction on the length of the run $r = c_0, c_1, \ldots, c_n$. Assume $|r| = 1$, namely r consists of the initial configuration c_0. Then all local states are empty, satisfying the Lemma. Assume the lemma holds for runs of length $|r| = n$, that c_n satisfies the Lemma, and consider the transition $c_n \to c_{n+1}$. If the transition is Mint and the claim holds for c_n, then after minting it still holds by adding the minted coin. If it is Swap, it still holds as Swap does not add new coins to the configuration or remove coins from it.

Grassroots.

Corollary 5.9. *Grassroots Cryptocurrencies are grassroots.*

PROOF. Consider $P \subset P'$ and a configuration $c \in C(P')$ such that $c/P \in C(P)$. This means that all coins held by members of P in c are P-coins. Hence the transaction in $R(P')$ in which a member of P swaps coins with a member of $P' \setminus P$ (possibly preceded by transactions in which the two members mint coins, if they have not done so already) satisfies the definition of interactivity, and GC is grassroots by Theorem 4.10.

5.3. Grassroots Federations via Joining and Leaving

Grassroots Federations [28] aim to address the democratic governance of large-scale decentralised digital communities, e.g., of the size of the EU, the US, existing social networks, and even humanity at large. A grassroots federations evolves via the grassroots formation and federation of digital communities, each governed by an assembly selected by sortition from its members. The approach assumes digital freedom of assembly—that people can freely form digital communities that can federate into ever-larger communities as well as spawn child communities, based on geography, relations, interests, or causes. Small communities (say < 100) are governed by their members. Larger communities—no matter how large—are each governed democratically by a small (say ≈ 100) assembly elected by sortition among its members.

A specification of grassroots federations via unary multiagent transition systems has yet to be attempted. It would require a grassroots consensus protocol by which members of two communities decide, atomically, to join or leave. Here, we define such atomic transactions abstractly: All members of a community carry its state locally; and all of them change its state atomically to realise joining or leaving another community. The details are shown below.

Background. Grassroots Federations [27, 28] is a process by which communities evolve and federate with other communities. Key steps include a community joining a federation, by the mutual consent of the two, and a community leaving a federation, by the unilateral decision of one of the two.

In the following $G = (V, E)$ is a federations graph with communities as nodes and directed edges indicating community relation, specifically $f \to v$

indicates that v is a child community of f, and G' is the graph resulting from applying any of the following transitions to G:

1. **Federate** $v \in V$**:** Add a new node $f \notin V$ to V and the edge $f \to v$ to E.
2. **Join** $f \to v$**:** Add $f \to v$ to E provided that $f \in V$ and G' is acyclic.
3. **Leave** $f \to v$**:** If $f \to v \in E$ then remove $f \to v$ from E.

Challenges. The grassroots solution would be to realise each community as a digital social contract [33], which also requires a grassroots consensus protocol among the participants, namely one that is executed by the participants themselves [34], not by third-parties (miners, validators) operating a global platform. In contrast to the undirected grassroots social graphs, grassroots federations graphs are directed. As a federation is intended to offer a hierarchical democratic governance structure, its directed graph must be acyclic. To allow local transactions to evolve the federation without creating cycles, a partial order $\succ$ on communities can be devised, with conditioning the joining of g as a child of f on $f \succ g$. Such a partial order can be defined, for example, based on height in the graph, geographic containment, or topic generality. In a real deployment of grassroots federations we expect the partial order to be a superposition of geography, topics, and maybe other issues, so that the federation of dog lovers of a village can join both the general village federation and the regional federation of dog lovers, which in turn can join the general regional federation. The resulting federations structure, termed *laminar*, was investigated in [27]. With this constraint we can address the requirement that the federations graph always be acyclic.

A related technical issue that arises in a grassroots setting is to ensure,

without central coordination, that each community has a globally-unique identifier. This is required so that a grassroots federations can scale indefinitely, integrating communities that have emerged independently of each other. To achieve that, any agent p may create one singleton community with itself as a member, identified by p. A community with a unique identifier c may create any number of parent communities using the **Federate** transition, and endows the ith parent community it creates with the unique identifier (c, i). Here, we employ community identifiers P^*, where each $c \in P^*$ consists of an agent $p \in P$ followed by a finite list of integers, and define a total (not partial) order $\succ$ on P^*, where $v \succ v'$ for $v, v' \in P^*$ if the list v is longer than the list v', with ties resolved lexicographically.

The fundamental difference between grassroots social graphs and grassroots federations is that the actors in the former are individual agents, whereas the actors in the latter are communities of agents. To address it, we provide an agent-based specification of grassroots federations: All members of a community hold an identical copy of its state, and a transaction between two communities, say g joining f, is realised by the corresponding k-ary atomic transaction among the members of the two communities, with k being the size of their union, in which the states of all members of both communities change atomically to reflect this federation-level transaction.

Naturally, this results in many aspects of the transaction being abstracted-away, including that:

1. Joining requires mutual consent among the two communities, and leaving is unilateral by one community.
2. Decisions on behalf of a community are taken by the assembly of a

community.

3. The decision process is constitutional and democratic.

Specification. A *federations graph* $G = (V, E)$ over P is a labelled directed acyclic graph with nodes labelled by community identifiers; we do not distinguish between nodes and their labels. The *initial federations graph* over P is $G_0(P) = (P, \emptyset)$. Let $\mathcal{G}(P)$ be the set of federations graphs over P.

Definition 5.10 (Community and Personal Subgraph). Given a federations graph $G = (V, E) \in \mathcal{G}(P)$, for a community $v \in V$ the **community subgraph** $G_v = (V_v, E_v)$ of v is the subgraph of G where V_v includes v and all nodes adjacent to v and E_v includes all edges incident with v. An agent $p \in P$ is a **member** of community $v \in G$ if there is a path in G from v to p, and the **personal subgraph** G_p of p is the union of the subgraphs of the communities p is a member of, namely $G_p := \bigcup_{v:p \in v} G_v$

Thus, G_p includes any community v that p is a member of and any edge incident with v.

Observation 5.11. *Let $G \in \mathcal{G}(P)$ for some set of agents $P \subset \Pi$. Then $G = \bigcup_{p \in P} G_p$.*

Proof (Proof of Observation 5.11). Let $G = (V, E)$ as above. We prove the two directions of equality:

1. $\subseteq$: Let $v \in V$. Then some $p \in P$ is a member of v by construction. Hence $v \in V_p$. Let $f \to g \in E$, with some $p \in V_f$. Then by definition $p \in V_g$, and hence $f \to g \in E_p$.

2. $\supseteq$ Let $p \in P$ and $v \in V_p$. Then p is a member of v in G, and therefore $v \in V$. Let $e \in E_p$. By construction, this can be the case only if $e \in E$.

Namely, a federations graph is fully-defined by its members' subgraphs. Hence, in the following grassroots federations transition system, the evolving federations graph G created during a run is stored in a distributed way, with each agent $p \in P$ maintaining G_p. The result is that each agent stores the state of all communities they are a member of; equivalently, the state of each community is stored by all its members. Formally:

Definition 5.12 (Federations Configuration, Valid). A **federations configuration** c over $P \subset \Pi$ has local states $\{G_p : G \in \mathcal{G}(P), p \in P\}$. The **initial federations** are $G_0 := (V_0, \emptyset) \in \mathcal{G}(P)$, where V_0 has a p-labelled node for every $p \in P$, and the **initial federations configuration** c_0 is defined by $c_{0,p} := G_{0,p} := (\{p\}, \emptyset)$. A federations configuration is **valid** if there is a graph $G \in \mathcal{G}(P)$ such that $c_p = G_p$ for every $p \in P$.

Note that the initial federations configuration is valid. The atomic transactions of grassroots federations ensure that when the state of a community v changes in a graph G through the addition or removal of an edge incident to v in G, this change is carried out atomically by all $p \in P_v$, each updating its local state G_p.

This formulation abstracts away the decision process. In practice, communities may employ assemblies selected by sortition [27, 28], which in turn may be responsible for storing the community social graph and employ a consensus protocol among them, rather than requiring storage and consensus from all members. Still, we use the direct-democracy model here to

simplify the exposition.

Transactions.

Definition 5.13 (Grassroots Federations Transactions). For agents $P \subset \Pi$ with local states being federations subgraphs from $\mathcal{G}(P)$, the grassroots federations transactions for every $G = (V, E) \in \mathcal{G}(P)$ are:

1. **Federate** v: c, c' are configurations over $Q := P_v$, $v \in V$, $f = (v, i+1)$, where i is the maximal index of a parent of $v \in G$, if any, $i = 0$ if none, and
   $\forall p \in Q.(c_p = G_p \wedge c'_p := (V \cup \{f\}, E \cup \{f \to v\})_p)$.
2. **Join** $f \to g$: c, c' are configurations over $Q := P_f \cup P_g$, $f, g \in V$, $f \succ g$, and
   $\forall p \in Q.(c_p = G_p \wedge c'_p := (V, E \cup \{f \to g\})_p)$.
3. **Leave** $f \to g$: c, c' are configurations over $Q := P_f$, $f, g \in V$, $f \to g \in E$, and $\forall p \in Q.(c_p = G_p \wedge c'_p := (V, E \setminus \{f \to g\})_p)$.

The participants of Federate v are the members of v; of Join $f \to g$, the members of f and of g; and of Leave $f \to g$, the members of f. In each, every participant is active: its personal subgraph gains or loses the edge the transaction adds or removes.

Definition 5.14 (Grassroots Federations). The **grassroots federations** GF is the transactions-based protocol (Definition 4.5) over the Federate, Join, and Leave transactions of Definition 5.13, with local-states function $S(P) := \{G_p : G \in \mathcal{G}(P), p \in P\}$ and initial local state $s_0(p) := (\{p\}, \emptyset)$ for every $p \in \Pi$.

Invariants.

Lemma 5.15 (Validity). *Given $P \subset \Pi$ and a run r of $GF(P)$, any configuration $c \in r$ is valid.*

PROOF. The proof is by induction on the length of the run. Initially, $G = G_0$ and the initial configuration c_0 satisfies $G_0 = \bigcup_{p \in P} c_{0,p}$. Assume that configuration c is valid and encodes $G = (V, E)$, and consider a transition $c \to c'$ with c' encoding G', namely $G' = \bigcup_{p \in P} c'_p$. We consider the various cases:

1. **Federate** v: Each $p \in P_v$ changes their state to $c'_p = (V \cup \{f\}, E \cup \{f \to v\})_p$.
2. **Join** $f \to g$: Each $p \in (P_f \cup P_g)$ changes their state to $c'_p = (V, E \cup \{f \to g\})_p$.
3. **Leave** $f \to g$: Each $p \in P_f$ changes their state to $c'_p = (V, E \setminus \{f \to g\})_p$.

Examining these changes shows that they all preserve validity, in that also in the new configuration each agent p records, by construction, exactly the p-projection of the updated federations graph G'.

Grassroots.

Corollary 5.16. *Grassroots Federations are grassroots.*

PROOF. Consider $P \subset P'$ and a configuration $c \in C(P')$ such that $c/P \in C(P)$. This means that the subgraph of G projected by members of P is a connected component of G. Hence the Join transaction in $R(P')$ in which

a member of P forms an edge with a member of $P' \setminus P$, the direction of which depends on the order $\succ$ of their identifiers, satisfies the definition of interactivity, and GF is grassroots by Theorem 4.10.

This completes the transactions-based specification of the three key grassroots platforms. It does not distinguish between transactions carried out by mutual consent (befriend, open a credit line, join), and transactions that are obligatory once requested (unfriend, redeem, leave). We discuss adding liveness requirements in Section 7.

6. Implementation

The specifications presented here provide the formal foundation for grassroots platforms. Recent work [32] explores implementing such platforms through Grassroots Logic Programs (GLP), a secure, multiagent, concurrent logic programming language. GLP provides binary communication primitives through paired reader/writer variables, where k-ary atomic transactions can be realised via consensus protocols such as Constitutional Consensus [34]. The notion of implementations among transition systems, as well as their fault-tolerance, has been studied extensively [29, 35–39]. Here, we discuss this notion briefly and informally, and plan follow-on work to do so formally.

Binary transactions. A standard way to realise binary transactions using unary transition systems is for one agent, say p, to OFFER the transaction to q, who may respond with ACCEPT, upon which p may respond with COMMIT, upon which the offered transaction is deemed to have been executed, or ABORT. Agent p may also issue ABORT before or after receiving any response from q to its offer, provided p has not previously issued COMMIT.

A challenge in this implementation is that a faulty p may fail to either COMMIT or ABORT following an ACCEPT by q, leaving q in limbo, at least in regards to this transaction. Solutions to this are a subject of future work.

For now, we note that, worst case, a friendship offer by p accepted by q, or an offer by community p to join community q, would remain in limbo. If it is committed by p at some later point, which is not convenient to q, then q can promptly unfriend p or remove p, as the case may be, with little or no harm done. In case of grassroots cryptocurrencies, a swap transaction in limbo may tie coins offered by q, which may or may not be harmful to q (not harmful if these are q-coins, which q may mint as it pleases; or p-coins that q tries to redeem, and if p is non-responsive it might indicate that p-coins are not worth much anyhow).

k-ary transactions. The k-ary transactions of grassroots federations require consensus among community members with support for reconfiguration as membership changes. Constitutional Consensus [34] provides such participant-controlled reconfiguration through constitutional amendments. More generally, grassroots platforms entail multiple dynamically-changing, partially-overlapping sets of agents engaged in running partially-dependent consensus protocols.

7. Related and Future Work

Atomic transactions have been investigated early in distributed computing, mostly in the context of database systems [40–42]. Most research since and until today focuses on their efficient and robust implementation [43, 44]. The integration of atomic transactions in programming languages has also

been explored [45]. In terms of formal models of concurrency, the extension of CCS with atomic transactions has been investigated in the past [46–48], but without follow-on research, so it seems. While transition systems have been the bedrock of abstract models of computation since the Turing machine, we are not aware of previous attempts to explore atomic transactions within their context.

Relation to the original definition of grassroots protocols. Previous work on formal implementations of grassroots platforms employed unary transition systems [24–26, 29]. While this new definition of a grassroots protocol expresses the same intuition as the original one [11], the new definition is crisper. It is also more restrictive in two senses, and more lax in a third:

1. It is specified in terms of configurations and transitions not runs, so its restriction applies to all configurations, not only those reachable via a run.
2. A technical limitation of comparing sets of runs, as done in the original definition, is that doing so does not account for the internal/hidden nondeterminism of intermediate configurations. So, according to the original definition, the smaller group P may have a run that leads to a configuration with multiple choices, while the larger group P' has a set of runs, each leading separately to only one of these choices of P, but no run of P' leads to a configuration in which the members of P have the same choices they would have if they were to run on their own. The current definition in terms of configurations and transitions, rather than sets of runs, eliminates this deficiency.
3. On the other hand, the original definition in terms of runs also ad-

dressed liveness. Within the original definition, an all-to-all dissemination protocol $\mathcal{F}$ in which $\mathcal{F}(P)$ satisfies the liveness requirement that every message sent by an agent in P eventually reaches every agent in P, is not grassroots. The reason is that a run of P, which is live when member of P run on their own, being oblivious of members outside P, would not be live within the context of P', as it would not provide dissemination between members of P and members of $P' \setminus P$, indefinitely. We consider this limitation of the new definition, as it relates to all-to-all dissemination, hypothetical, as it seems that such a protocol could not be realised without global directory (e.g., a DHT) for agents to find each other, which would not be oblivious, and hence not grassroots also under the new definition.

The two definitions can be compared where both apply: on protocols all of whose transitions are unary and which carry no liveness requirement. In [11], a protocol assigns to every P a transition system $TS(P) = (P, S(P), T(P), \lambda(P))$ over a local-states function S with a single initial local state s_0, whose transitions are p-transitions—each changes the local state of one agent $p \in P$—and $\lambda(P)$ is its liveness requirement, a collection of sets of transitions; the *projection* of $TS(P')$ on $P \subset P'$ is $TS(P')/P := (P, S(P'), T(P')/P, \lambda(P')/P)$ with $T(P')/P := \{c_1/P \to c_2/P : c_1 \to c_2 \in T(P')\}$; $TS \subseteq TS'$ if the local states, the transitions and the liveness requirement of TS are included in those of TS', the last restricted to the transitions of TS; and a protocol is grassroots if $TS(P) \subset TS(P')/P$ for every $\emptyset \subset P \subset P' \subseteq \Pi$. A protocol in the sense of Definition 4.1 all of whose transitions are unary, with the same initial local state s_0 for every agent and with $\lambda(P) := \emptyset$ for every P, is

therefore also a protocol in the sense of [11]; we call such a protocol *unary*. On unary protocols the new definition is the stronger, and strictly so, even when only reachable configurations are considered:

Proposition 7.1 (Relation to the original definition). *A unary protocol that is grassroots by Definition 4.3 is grassroots in the sense of [11], and not conversely.*

PROOF. Let $\mathcal{F}$ be a unary protocol, grassroots by Definition 4.3, and $\emptyset \subset P \subset P' \subseteq \Pi$. The local states satisfy $S(P) \subset S(P')$ by the definition of a local-states function, and the liveness requirements are empty on both sides. For the transitions, let $t = c_1 \to c_2 \in T(P)$, and let c'_1, c'_2 be the configurations over P' with $c'_{i,p} := c_{i,p}$ for $p \in P$ and $c'_{i,q} := s_0$ for $q \in P' \setminus P$, $i = 1, 2$. Then $c'_1 \to c'_2 \in t{\uparrow}P'$ by Definition 3.3, hence $c'_1 \to c'_2 \in T(P) \uparrow P' \subseteq T(P')$ by obliviousness, and its projection on P is t; so $T(P) \subseteq T(P')/P$, and $TS(P) \subseteq TS(P')/P$. For strictness, apply interactivity to the initial configuration $c_0(P')$, whose projection on P is $c_0(P) \in C(P)$: there is a computation $c_0(P') \xrightarrow{*} c'$ of $\mathcal{F}(P')$ with $c'/P \notin C(P)$. Let $d \to d'$ be its first transition with $d'/P \notin C(P)$, which exists since $c_0(P')/P \in C(P)$. Then $d/P \to d'/P \in T(P')/P$, but $d/P \to d'/P \notin T(P)$, as $d'/P \notin C(P) = S(P)^P$ while every transition in $T(P)$ is between configurations in $C(P)$. Hence $T(P) \subset T(P')/P$, the inclusion being strict already on the transitions, and $\mathcal{F}$ is grassroots in the sense of [11].

For the converse, consider the protocol $\mathcal{F}$ over $S(P) := 2^P$ with $s_0 := \emptyset$ in which every agent may greet one other agent, once: $\mathcal{F}$ is the transactions-based protocol over the unary transactions $\mathrm{greet}(p, q)$, for $p \neq q \in \Pi$, with

participant p, where $c'_p := \{q\}$ provided $c_p = \emptyset$. It is oblivious by Proposition 4.6. Let $\emptyset \subset P \subset P' \subseteq \Pi$. Then $T(P) \subseteq T(P')/P$, by the argument of the first part, which used only obliviousness; and for $p \in P$ and $q \in P' \setminus P$ the projection on P of the transition of $\mathcal{F}(P')$ in which p greets q is in $T(P')/P$ and not in $T(P)$, its target having $\{q\} \notin S(P)$ as the local state of p. Hence $\mathcal{F}$ is grassroots in the sense of [11]. It is not interactive by Definition 4.3: let $P = \{p_1, p_2\}$, $P' \supset P$, and c the configuration of $\mathcal{F}(P')$ reached from $c_0(P')$ by p_1 greeting p_2 and p_2 greeting p_1. Then $c/P \in C(P)$, but no transition of $\mathcal{F}(P')$ changes the local state of p_1 or of p_2, both being nonempty, so every computation $c \xrightarrow{*} c'$ of $\mathcal{F}(P')$ has $c'/P = c/P \in C(P)$. Hence $\mathcal{F}$ is not grassroots by Definition 4.3.

The example shows the first of the two senses in which the new definition is more restrictive: it requires the ability to interact at every configuration, where the original requires only that some transition of the larger group not be available to the smaller one. Where a liveness requirement is present the two definitions are not comparable, as item 3 above discusses; the framework of this paper has none.

Liveness. While liveness is well-understood in the context of unary transition systems, it is opaque in the more abstract case of atomic transactions, as it may not be possible to identify the culprit in case an enabled transaction is never taken. On the other hand, a unary implementation of atomic transactions clearly has liveness requirements. Hence, we opted not to incorporate liveness in the current context of atomic transactions, and leave it as a subject of future research.

The direction we envision is viewing every agent as a pair: a nondeter-

ministic *user* and a deterministic *app*, with a *user interface* providing bidirectional communication transactions between the user and the app [32]. As the nondeterministic user models a volitional person, liveness requirements in such a system may take the form "a message sent by a person p to their friend q is eventually received by q" [24], or "a swap request from p to q that includes q-coins is eventually responded to by q" [26].

Subsequent work. Several of the directions named above have since been taken up. Volition-guarded multiagent atomic transactions [30] extend the transactions of this paper with the volitions of the people operating the machines, decomposing each agent's state into a machine state and a volitional state, incorporate liveness into the definition of a grassroots protocol, recast there in terms of the interleaving of correct runs of two disjoint groups of agents, and adapt to that setting the notions of implementation and of fault-resilient implementation; the two definitions of grassroots are not comparable in general, neither implying the other [30]. Grassroots protocols are placed within a taxonomy of centralised, decentralised, federated and grassroots platforms in [49]. The passage from a social contract written as legal text, through act schemas, to volition-guarded transactions is the subject of [50]. None of these results is used here; this paper stands on the definitions of Sections 2–4.

Acknowledgements

I thank Idit Keidar, Andy Lewis, and Nimrod Talmon for our discussions and their feedback on this and related topics. Useful discussions with Claude and Chat-GPT are acknowledged.

Funding

This research did not receive any specific grant from funding agencies in the public, commercial, or not-for-profit sectors.

CRediT authorship contribution statement

Ehud Shapiro: Conceptualization, Methodology, Formal analysis, Investigation, Writing – original draft, Writing – review & editing.

Declaration of generative AI and AI-assisted technologies in the manuscript preparation process

During the preparation of this work I used Claude (Anthropic) and ChatGPT (OpenAI) in order to revise the text and check the proofs. After using these tools, I reviewed and edited the content as needed and take full responsibility for the content of the published article.

References

[1] E. Shapiro, Grassroots platforms with atomic transactions: Social graphs, cryptocurrencies, and democratic federations, in: Proceedings of the 27th International Conference on Distributed Computing and Networking, 2026, pp. 71–81, arXiv preprint arXiv:2502.11299. doi:10.1145/3772290.3772309.

[2] S. Zuboff, The age of surveillance capitalism: The fight for a human future at the new frontier of power, Public Affairs, US, 2019.
URL https://www.hachettebookgroup.com/titles/

shoshana-zuboff/the-age-of-surveillance-capitalism/
9781610395694/

[3] S. Zuboff, Surveillance capitalism or democracy? the death match of institutional orders and the politics of knowledge in our information civilization, Organization Theory 3 (3) (2022) 26317877221129290. doi:10.1177/26317877221129290.

[4] Ethereum, Decentralized autonomous organizations (DAOs) — ethereum.org (2021).
URL https://ethereum.org/en/dao/

[5] Y. Faqir-Rhazoui, J. Arroyo, S. Hassan, A comparative analysis of the platforms for decentralized autonomous organizations in the ethereum blockchain, Journal of Internet Services and Applications 12 (2021) 1–20. doi:10.1186/s13174-021-00139-6.

[6] S. Nakamoto, A. Bitcoin, A peer-to-peer electronic cash system, Bitcoin.–URL: https://bitcoin. org/bitcoin. pdf 4 (2008).

[7] G. Wood, et al., Ethereum: A secure decentralised generalised transaction ledger, Ethereum project yellow paper 151 (2014) (2014) 1–32.
URL https://ethereum.github.io/yellowpaper/paper.pdf

[8] S. Wang, Y. Yuan, X. Wang, J. Li, R. Qin, F.-Y. Wang, An overview of smart contract: architecture, applications, and future trends, in: 2018 IEEE Intelligent Vehicles Symposium (IV), IEEE, 2018, pp. 108–113. doi:10.1109/ivs.2018.8500488.

[9] Z. Wang, H. Jin, W. Dai, K.-K. R. Choo, D. Zou, Ethereum smart contract security research: survey and future research opportunities, Frontiers of Computer Science 15 (2) (2021) 1–18.

[10] V. Buterin, Governance, part 2: Plutocracy is still bad, available at https://vitalik.eth.limo/general/2018/03/28/plutocracy.html (2018).
URL https://vitalik.eth.limo/general/2018/03/28/plutocracy.html

[11] E. Shapiro, Grassroots distributed systems: Concept, examples, implementation and applications (brief announcement), in: 37th International Symposium on Distributed Computing (DISC 2023). (Extended version: arXiv:2301.04391), LIPICS, Italy, 2023, pp. 47:1, 47:7.
URL https://arxiv.org/abs/2301.04391

[12] E. Shapiro, A grassroots architecture to supplant global digital platforms by a global digital democracy, arXiv:2404.13468, Proceedings of DAWO'24 (2024).
URL https://arxiv.org/abs/2404.13468

[13] P.-J. Proudhon, J. B. Robinson, General idea of the revolution in the nineteenth century, Courier Corporation, 2004, original work published 1851.

[14] S. Nakamoto, A. Bitcoin, A peer-to-peer electronic cash system (2008).

[15] J. Benet, Ipfs-content addressed, versioned, p2p file system, arXiv preprint arXiv:1407.3561 (2014).

[16] P. Maymounkov, D. Mazières, Kademlia: A peer-to-peer information system based on the xor metric, in: Peer-to-Peer Systems: First International Workshop, IPTPS 2002, Springer, Cambridge, MA, USA, 2002, pp. 53–65. doi:10.1007/3-540-45748-8_5.

[17] G. Chockler, R. Melamed, Y. Tock, R. Vitenberg, Constructing scalable overlays for pub-sub with many topics, in: Proceedings of the twenty-sixth annual ACM symposium on Principles of distributed computing, 2007, pp. 109–118.

[18] G. Chockler, R. Melamed, Y. Tock, R. Vitenberg, Spidercast: a scalable interest-aware overlay for topic-based pub/sub communication, in: Proceedings of the 2007 inaugural international conference on Distributed event-based systems, 2007, pp. 14–25.

[19] S. Buchegger, D. Schiöberg, L.-H. Vu, A. Datta, Peerson: P2p social networking: early experiences and insights, in: Proceedings of the Second ACM EuroSys Workshop on Social Network Systems, 2009, pp. 46–52. doi:10.1145/1578002.1578010.

[20] A. Raman, S. Joglekar, E. D. Cristofaro, N. Sastry, G. Tyson, Challenges in the decentralised web: The mastodon case, in: Proceedings of the internet measurement conference, 2019, pp. 217–229. doi:10.1145/3355369.3355572.

[21] TorrentFreak, BitTorrent Turns 20: The File-Sharing Revolution Revisited, contains Bram Cohen's original statement: "BitTorrent's customer is etree. Etree is a loose-knit community of people who

994 distribute live concert recordings online" (7 2021).
995 URL https://torrentfreak.com/bittorrent-turns-20-the-file-sharing-revolution-

996 [22] D. Tarr, E. Lavoie, A. Meyer, C. Tschudin, Secure scuttlebutt: An
997 identity-centric protocol for subjective and decentralized applications,
998 in: Proceedings of the 6th ACM conference on information-centric net-
999 working, 2019, pp. 1–11. doi:10.1145/3357150.3357396.

1000 [23] A.-M. Kermarrec, E. Lavoie, C. Tschudin, Gossiping with append-only
1001 logs in secure-scuttlebutt, in: Proceedings of the 1st international work-
1002 shop on distributed infrastructure for common good, 2020, pp. 19–24.
1003 doi:10.1145/3428662.3428794.

1004 [24] E. Shapiro, Grassroots social networking: Serverless, permissionless pro-
1005 tocols for twitter/linkedin/whatsapp, in: OASIS '23, Association for
1006 Computing Machinery, 2023. doi:10.1145/3599696.3612898.

1007 [25] E. Shapiro, Grassroots currencies: A foundation for a grassroots digital
1008 economy, arXiv preprint arXiv:2202.05619 (2022).
1009 URL https://arxiv.org/abs/2202.05619

1010 [26] A. Lewis-Pye, O. Naor, E. Shapiro, Grassroots flash: A payment system
1011 for grassroots cryptocurrencies, arXiv preprint arXiv:2309.13191 (2023).
1012 URL https://arxiv.org/abs/2309.13191

1013 [27] D. Halpern, A. D. Procaccia, E. Shapiro, N. Talmon, Federated assem-
1014 blies, Proc AAAI 2025; arXiv preprint arXiv:2405.19129 (2024).
1015 URL https://arxiv.org/abs/2405.19129

[28] N. Talmon, E. Shapiro, Grassroots federation: Fair democratic governance at scale, arXiv preprint arXiv:2505.02208; also Proc. of AAMAS'26 (2025).
URL https://arxiv.org/abs/2505.02208

[29] E. Shapiro, Multiagent transition systems: Protocol-stack mathematics for distributed computing, arXiv preprint arXiv:2112.13650 (2021).
URL https://arxiv.org/abs/2112.13650

[30] A. Lewis-Pye, E. Shapiro, Volition-guarded multiagent atomic transactions: Describing people and their machines, arXiv preprint arXiv:2604.25596 (2026).
URL https://arxiv.org/abs/2604.25596

[31] Project Liberty, Decentralized social networking protocol (dsnp) specification, https://dsnp.org/, version 1.0 (2021).

[32] E. Shapiro, Glp: A grassroots, multiagent, concurrent, logic programming language for ai (full version), arXiv preprint arXiv:2510.15747, Summary in Proc. of ICLP'26, EPTCS 450, pp. 119–133 (2025).
URL https://arxiv.org/abs/2510.15747

[33] L. Cardelli, L. Orgad, G. Shahaf, E. Shapiro, N. Talmon, Digital social contracts: A foundation for an egalitarian and just digital society, in: CEUR Proceedings of the First International Forum on Digital and Democracy, Vol. 2781, CEUR-WS, 2020, pp. 51–60.
URL https://ceur-ws.org/Vol-2781/paper5.pdf

[34] I. Keidar, A. Lewis-Pye, E. Shapiro, Constitutional consensus, arXiv preprint arXiv:2505.19216 (2025).
URL https://arxiv.org/abs/2505.19216

[35] M. Abadi, L. Lamport, Composing specifications, ACM Transactions on Programming Languages and Systems (TOPLAS) 15 (1) (1993) 73–132.

[36] L. Lamport, Specifying concurrent systems with tla$^+$, NATO ASI SERIES F COMPUTER AND SYSTEMS SCIENCES 173 (1999) 183–250.

[37] P. B. Menezes, J. F. Costa, A. Sernadas, Refinement mapping for general (discrete event) systems theory, in: International Conference on Computer Aided Systems Theory, Springer, 1995, pp. 103–116.

[38] P. Manolios, A compositional theory of refinement for branching time, in: Advanced Research Working Conference on Correct Hardware Design and Verification Methods, Springer, 2003, pp. 304–318.

[39] J. R. Wilcox, D. Woos, P. Panchekha, Z. Tatlock, X. Wang, M. D. Ernst, T. Anderson, Verdi: a framework for implementing and formally verifying distributed systems, in: Proceedings of the 36th ACM SIGPLAN Conference on Programming Language Design and Implementation, 2015, pp. 357–368.

[40] B. W. Lampson, Chapter 11. atomic transactions, in: Distributed Systems—Architecture and Implementation: an Advanced Course, Springer, 1981, pp. 246–265.

[41] N. A. Lynch, M. Merritt, Atomic transactions: in concurrent and distributed systems, Morgan Kaufmann Publishers Inc., 1993.

[42] N. Lynch, M. Merritt, W. Weihl, A. Fekete, A theory of atomic transactions, in: ICDT'88: 2nd International Conference on Database Theory Bruges, Belgium, August 31–September 2, 1988 Proceedings 2, Springer, 1988, pp. 41–71.

[43] M. Bravo, A. Gotsman, Reconfigurable atomic transaction commit, in: Proceedings of the 2019 ACM Symposium on Principles of Distributed Computing, 2019, pp. 399–408.

[44] G. Chockler, A. Gotsman, Multi-shot distributed transaction commit, Distributed Computing 34 (2021) 301–318.

[45] J. Borgström, K. Bhargavan, A. D. Gordon, A compositional theory for stm haskell, in: Proceedings of the 2nd ACM SIGPLAN Symposium on Haskell, 2009, pp. 69–80.

[46] L. Acciai, M. Boreale, S. Dal Zilio, A concurrent calculus with atomic transactions, in: European Symposium on Programming, Springer, 2007, pp. 48–63.

[47] E. de Vries, V. Koutavas, M. Hennessy, Communicating transactions, in: International Conference on Concurrency Theory, Springer, 2010, pp. 569–583.

[48] E. De Vries, V. Koutavas, M. Hennessy, Liveness of communicating transactions, in: Asian Symposium on Programming Languages and Systems, Springer, 2010, pp. 392–407.

[49] E. Shapiro, Characterising global platforms: Centralised, decentralised,

federated, and grassroots, arXiv preprint arXiv:2511.03286 (2025).
URL https://arxiv.org/abs/2511.03286

[50] J. Golike, A. Lewis-Pye, E. Shapiro, Formalising grassroots social contracts: From legal text to grassroots platforms, to appear on arXiv (2026).